ASSIGNMENT 1

Design, Implement and Validate a Reliable Billing Program
for a Rural Electricity Co-operative

Course Outcome	CO1 – Structured C programming
Subject	C Programming
Student Name	_____
Register Number	_____
Department	B.Tech – ECE
Assignment Type	Individual

Submitted with source code, test cases, expected outputs and design analysis.

Scenario and tariff used throughout

Grama Jyothi is a rural electricity co-operative. The original meter is a 16-bit counter. The monthly bill has a fixed charge of Rs. 50.00 and these energy slabs:

Consumption slab	Energy rate
First 100 units	Free
Next 100 units (101–200)	Rs. 2.25 per unit
Above 200 units	Rs. 4.50 per unit
Fixed charge	Rs. 50.00 per bill

Important unit convention: 1 electricity unit = 1 kWh. Money is stored as integer paise (100 paise = Rs. 1), so calculations do not accumulate floating-point rounding errors.

Billing formula: total bill (paise) = 5000 + energy charge (paise). Energy charge = 0 for the first 100 units, 225 paise for each unit from 101 to 200, and 450 paise for each unit above 200.

TASK 1 – Understand the Problem and Identify the Data (4 Marks)

1. Inputs and outputs

- **Previous meter reading:** the counter value recorded at the previous billing time. It is an integer in the range 0–65535 for the original 16-bit meter.
- **Current meter reading:** the latest counter value, also 0–65535. If it is smaller than the previous reading, the counter may have rolled over.
- **Household number:** an identifier used to associate a bill with a customer. It is not used in the arithmetic; store it as an unsigned integer or a character string if leading zeroes are meaningful.
- **Category:** customer/tariff category, if the co-operative later introduces different tariffs. Use an enum or validated integer code.
- **Outputs:** units consumed, energy charge, fixed charge and total bill. Units are whole units for the original integer meter; money is represented in paise and displayed as rupees and paise.

2. Constants and variable values

- The slab limits (100 and 200 units), slab rates (0, 225 and 450 paise/unit), fixed charge (5000 paise), and meter modulus (65536) are tariff/system constants for the original design.
- Readings, household number, units consumed and calculated bill are variables because they differ for each customer and billing cycle.

3. Negative, fractional and large values

- A valid meter reading cannot be negative. A reading is not fractional in the original meter. Units and total money can become large when many households are processed. The total bill must never be negative.
- A bill may contain fractional rupees, such as Rs. 162.50, so integer paise is the suitable representation. The original 16-bit reading can roll over; a wider yearly or district total is required.

4. Correctness rules and assumptions

- The total must equal fixed charge plus energy charge. A current reading below the previous reading is treated as one 16-bit rollover, provided the meter has not reset or been replaced.
- Assume one rollover at most between readings, the tariff applies to all households, every meter reading is valid, and each billing period has one previous and one current reading. In production, meter replacement/reset must be recorded separately.

Alternative representation: readings stored as text

A text reading such as "01530" preserves leading zeroes but must be converted before arithmetic. Converting a string of d digits takes O(d) time by scanning its characters and uses O(d) input space (or O(1) extra space for a streaming conversion). An integer reading is already numeric, so comparison and subtraction take O(1) time and the stored value uses a fixed number of bytes. Text is useful for preserving formatting and validating input, but integer storage is simpler and faster for meter arithmetic.

Evidence: for a 5-digit reading such as 01530, conversion scans five characters; numeric subtraction then takes constant time.

TASK 2 – Choose the Data Types (4 Marks)

1. Data type selection

Data item	Suggested type	Reason
Meter reading	unsigned short	16-bit meter range 0–65535 on the target system; verify with sizeof.
Household ID	unsigned int / char[]	Numeric ID, or string if leading zeroes/letters occur.
Units / cumulative units	unsigned long long	Avoid overflow when many readings or periods are accumulated.
Money in paise	long long	Exact integer arithmetic; supports large totals.
Tariff rate	const long / int	Whole paise per unit; exact values.
Category	enum	Readable named categories.

The exact sizes of short, int and long are implementation-dependent. Use sizeof and limits from limits.h / stdint.h; do not assume that int is always 32 bits. For portable fixed-width values, use uint16_t and uint64_t from stdint.h when the implementation provides them.

2. Why not float for money?

Binary floating-point cannot represent many decimal fractions exactly. Repeated additions can produce tiny rounding errors. Integer paise represents Rs. 162.50 exactly as 16250 paise. For example, adding a floating-point value such as 0.1 repeatedly may not produce an exact decimal total; integer paise avoids this problem.

3. 16-bit rollover and the given program

An unsigned 16-bit value has 65536 possible values, 0 through 65535. Unsigned arithmetic wraps modulo 65536. Thus, after 65535 the next value is 0. For previous = 65400 and current = 120, the consumption is (120 – 65400) modulo 65536 = 256 units, not a negative number.

```
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    /* Unsigned subtraction wraps modulo 65536 on a 16-bit meter. */
    return (unsigned short)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned int)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));
    return 0;
}
```

Expected output on a system with 16-bit unsigned short:

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

The second result is 256 because unsigned subtraction is performed modulo 65536. The result is converted to the unsigned 16-bit range, so the wrap-around difference is represented as a positive value.

4. Float error across 4,000 bills

If each bill is accumulated using float rupees, small representation errors can add up over 4,000 additions. Integer paise gives an exact sum as long as the chosen integer type does not overflow. Use long long for the co-operative total and check the maximum expected total.

Alternative: double rupees

Using double rupees generally provides much better precision than float, and each addition is O(1) time with O(1) storage per bill. However, binary decimal fractions are still not always exact and a rounding rule is required when converting to paise. Integer paise also uses O(1) time and space per bill and is exact for the defined currency precision. Therefore, integer paise is preferred.

TASK 3 – Define the Constants (3 Marks)

1. Constants and naming

Use uppercase names with underscores for macros, such as SLAB1_UNITS and FIXED_CHARGE_PAISE. Keep tariff values in one place so a revision changes every calculation consistently.

- **#define:** preprocessor text replacement; no variable storage and no type checking.
- **const variable:** a typed object that cannot be modified through that name; it has scope and may occupy storage.
- **enum:** suitable for a set of named integer choices, such as customer categories or tariff modes; not ideal for storing arbitrary monetary rates.

```
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 100
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

/* Returns the bill in paise. */
long compute_bill(unsigned long units)
{
    long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB1_UNITS)
    {
        unsigned long n = units - SLAB1_UNITS;
        if (n > SLAB2_UNITS) n = SLAB2_UNITS;
        paise += (long)n * RATE2_PAISE;
    }
    if (units > SLAB1_UNITS + SLAB2_UNITS)
        paise += (long)(units - SLAB1_UNITS - SLAB2_UNITS)
                 * RATE3_PAISE;

    return paise;
}

int main(void)
{
    unsigned long tests[] = {0, 100, 150, 200, 350};
    int i;
    for (i = 0; i < 5; i++)
    {
        long p = compute_bill(tests[i]);
        printf("%3lu units -> Rs %ld.%02ld\\n",
               tests[i], p / 100, p % 100);
    }
    return 0;
}
```

Expected output:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

Changing RATE2_PAISE from 225 to 230 changes all bills using the second slab without editing the calculation statements. For 150 units, the new bill is Rs. 165.00; for 200 units it is Rs. 280.00.

Alternative: const or enum

Replacing macros with const variables adds typed declarations and may use storage, depending on the compiler and use. An enum is appropriate for named integer states but can make units/rates less clear. All approaches are O(1) per bill; the main difference is type checking, scope, storage and maintainability. Macros are convenient for compile-time constants, while const variables are generally safer where a typed object is useful.

TASK 4 – Use Macros Safely (4 Marks)

1. Safe macro examples

```
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define RUPEES_TO_PAISE(r) ((long long)(r) * 100LL)

/* Examples */
int x = SQUARE(a + 1);
int largest = MAX(i, j);
long long p = RUPEES_TO_PAISE(50);
```

- Parentheses around every parameter and the entire expression prevent precedence errors. Without them, SQUARE(a+1) written as x*x could expand to a+1*a+1, which is not (a+1) squared.
- MAX(a,b) can evaluate an argument more than once. MAX(i++,j) may increment i twice. Do not pass expressions with side effects to such macros.
- A function is better when type checking, one-time argument evaluation, debugging and clearer error messages are important.

2. Preprocessor evidence using gcc -E

```
gcc -E macros.c -o macros.i
```

The preprocessed file shows macro calls replaced by their expanded expressions. For example, SQUARE(a + 1) becomes ((a + 1) * (a + 1)). Include this generated file or a short screenshot in the submission as evidence.

Alternative: static inline functions

An inline function has type checking and evaluates each argument once. A function call may have a small call overhead if the compiler does not inline it, while a macro is expanded at compile time. For N uses, macro expansion can increase code size by roughly O(N × expression size); an inline function may share one function body or be inlined by the optimizer. Both are effectively O(1) for a small fixed expression, but functions are safer for complex operations.

```
static inline int square_int(int x) { return x * x; }
static inline int max_int(int a, int b) { return a > b ? a : b; }
```

TASK 5 – Organise the Program with Header Files and Conditional Compilation (3 Marks)

1. Program structure

A C program normally contains header inclusions, macro/constant definitions, type declarations, function prototypes, main(), and function definitions. Put shared declarations in a header file and implementation in a .c file.

```
/* billing.h */
#ifndef BILLING_H
#define BILLING_H

#define FIXED_CHARGE_PAISE 5000L
long compute_bill(unsigned long units);
unsigned int consumption(unsigned short previous,
                         unsigned short current);

#endif

/* billing.c */
#include <stdio.h>
#include "billing.h"
```

```
unsigned int consumption(unsigned short previous,
                         unsigned short current)
{
    return (unsigned short)(current - previous);
}

long compute_bill(unsigned long units)
{
    long p = FIXED_CHARGE_PAISE;
    if (units > 100) {
        unsigned long n = units - 100;
        if (n > 100) n = 100;
        p += (long)n * 225;
    }
    if (units > 200) p += (long)(units - 200) * 450;
    return p;
}
```

2. Include guards and include forms

The #ifndef / #define / #endif guard prevents the same header from being processed more than once in one translation unit, avoiding duplicate declarations. #include searches the compiler's configured system include paths. #include "file" searches the current/source directory first and then configured include paths, which is convenient for project headers.

3. Debug-only output

```
#ifdef DEBUG
#define DEBUG_PRINT(...) printf(__VA_ARGS__)
#else
#define DEBUG_PRINT(...) ((void)0)
#endif

/* Example */
DEBUG_PRINT("units=%lu bill=%ld\n", units, bill);
```

Compile normally with gcc billing.c -o billing. Compile with debugging output using gcc -DDEBUG billing.c -o billing_debug. With #ifdef DEBUG, the debug statement is removed during preprocessing in a normal build. A run-time if(debug) check remains in the executable and performs a condition test each time the statement is reached; the compile-time method avoids that repeated test and can remove debug strings/code from a release build.

TASK 6 – Read and Print Data Correctly (4 Marks)

1. Format specifiers

Type	printf / scanf format
int	%d
unsigned int	%u
long	%ld
unsigned long	%lu
long long	%lld
float	%f (printf); %f (scanf)
double	%f (printf); %lf (scanf)
char	%c
string (char array)	%s

A mismatch between the format specifier and variable type causes undefined behaviour in variadic functions such as printf. For example, using %d for a long is not correct. Use the exact matching specifier and cast only when the conversion is intentional.

2. Input validation

```
#include <stdio.h>

int main(void)
{
    unsigned int previous, current;
    int converted;
```

```
    printf("Enter previous and current readings: ");
    converted = scanf("%u %u", &previous, &current);

    if (converted != 2) {
        printf("Invalid input: enter two unsigned integers.\\n");
        return 1;
    }

    printf("Readings: %u and %u\\n", previous, current);
    return 0;
}
```

scanf returns the number of successfully assigned input items. If the expected count is not returned, the input is invalid. A production program should also reject values outside the meter's allowed range and clear or discard invalid input before asking again. A negative sign must not be accepted for an unsigned meter reading.

3. Print rupees and paise

```
long long bill_paise = 16250;
printf("Bill = Rs. %lld.%02lld\\n",
       bill_paise / 100, bill_paise % 100);
```

Output: Bill = Rs. 162.50. The quotient gives whole rupees and the remainder gives paise, with two digits after the decimal point.

Alternative: fgets and conversion

With fgets, the program reads a line of length L in O(L) time and uses O(L) input-buffer space. strtoul/sscanf can then convert and validate the line. scanf is concise and uses constant extra storage for fixed fields, but invalid input can remain in the input stream and requires careful recovery. fgets gives better control over complete-line validation.

TASK 7 – Control Type Conversion and Precision (4 Marks)

1. Conversion points

- Integer promotion: small integer types such as unsigned short are promoted before arithmetic. The result may then be converted back to the destination type.
- Integer division: 150 / 100 produces 1, not 1.5. Use a floating operand only when a fractional result is genuinely needed.
- Explicit casts: converting a large long to int can lose information if the value is outside the int range.
- Rounding: when converting fractional rupees to paise, choose one consistent rule, such as rounding to the nearest paise. Integer paise avoids repeated binary floating-point rounding.

2. Three examples of unexpected results

```
printf("%d\\n", 150 / 100);          /* 1, fractional part lost */
printf("%.2f\\n", (double)150 / 100); /* 1.50 */
unsigned short r = (unsigned short)(120 - 65400);
printf("%u\\n", (unsigned int)r);     /* 256: modulo 65536 */
int small = (int)5000000000LL;        /* may not fit in int */
```

The first expression uses integer division. The second converts one operand to double before division. The third intentionally uses 16-bit unsigned wrap-around. The fourth is implementation-dependent when the value cannot be represented by int; it must not be used for large totals. Use long long or uint64_t instead.

Alternative: floating-point calculations

If all bills are calculated in float and rounded only at the end, the total can differ slightly from the exact paise sum because floating-point operations are approximate. Error may accumulate with the number of bills; the exact size depends on values and operation order. Integer paise gives exact additions and O(1) time per bill, with O(1) storage per accumulator. Floating-point also uses O(1) time and space per addition but requires a documented rounding step. For financial billing, integer paise is the consistent choice.

TASK 8 – Trace the Compilation Process (4 Marks)

1. Compilation stages

Command	Output / purpose
gcc -E billing.c -o billing.i	Preprocessed C; headers expanded and macros replaced.
gcc -S billing.c -o billing.s	Assembly source generated.
gcc -c billing.c -o billing.o	Object file generated; not yet linked.
gcc billing.o -o billing	Links object code and libraries into executable.

2. Errors and warnings

- **Compile-time error:** syntax/type error, such as a missing semicolon or undeclared variable. The compiler cannot produce valid object code.
- **Link-time error:** a function is declared and called but no matching definition is linked; the linker reports an undefined reference.
- **Run-time error:** a problem while the program runs, such as invalid input or division by zero.
- Compile with gcc -Wall -Wextra -std=c11 billing.c -o billing. Resolve warnings by using matching format specifiers, initializing variables, checking function return values and removing unused variables.

3. What preprocessing changes

The preprocessor expands #include directives, substitutes macros, and includes or excludes code controlled by #if/#ifdef. It does not perform the later machine-code generation stages.

Alternative: multiple source files

With input.c, tariff.c and report.c plus a shared header, each .c file is compiled separately into an object file and then linked. If one source file changes, only that file normally needs recompilation, followed by linking. In a single-file program, the whole source is recompiled after a change. For a small fixed project the difference is minor; for a large project separate compilation saves build time. Object-file storage is the sum of the sizes of the individual .o files plus shared metadata.

TASK 9 – Implement and Test the Billing Program (4 Marks)

1. Complete original billing program

```
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    return (unsigned short)(current - previous);
}

long long compute_bill_paise(unsigned long long units)
{
    long long bill = 5000; /* fixed charge */
    if (units > 100) {
        unsigned long long n = units - 100;
        if (n > 100) n = 100;
        bill += (long long)n * 225;
    }
    if (units > 200)
        bill += (long long)(units - 200) * 450;
    return bill;
}

void print_bill(unsigned long long units)
{
    long long p = compute_bill_paise(units);
    printf("%llu units: Rs. %lld.%02lld\\n",
           units, p / 100, p % 100);
}

int main(void)
{
    unsigned long long tests[] = {0, 100, 150, 200, 350};
    int i;
```

```
    printf("Maximum unsigned short = %u\\n", USHRT_MAX);
    for (i = 0; i < 5; ++i) print_bill(tests[i]);

    printf("Rollover consumption: %u units\\n",
           consumption((meter_t)65400, (meter_t)120));
    return 0;
}
```

2. Test table

Test	Input / condition	Expected result	Status
Normal slab 1	0 units	Rs. 50.00	Pass
Boundary	100 units	Rs. 50.00	Pass
Slab 2	150 units	Rs. 162.50	Pass
Boundary	200 units	Rs. 275.00	Pass
Slab 3	350 units	Rs. 950.00	Pass
Rollover	65400 → 120	256 units	Pass
Invalid input	letters / negative reading	Reject input	Validation required
Current < previous	without rollover event	Flag possible reset/replacement	Validation required

The first six cases can be verified directly using the program. Invalid input and suspicious decreases must be rejected or flagged by the input-validation layer; the rollover calculation alone cannot distinguish a genuine rollover from a meter reset. A real implementation should record a rollover/reset event.

3. Five-bill hand calculation example

For readings that yield 50, 100, 150, 200 and 350 units, the bills are Rs. 50.00, Rs. 50.00, Rs. 162.50, Rs. 275.00 and Rs. 950.00. Total = Rs. 1,487.50. This is the expected sum for the five-bill sample.

Alternative: slab arrays and a loop

Store slab limits and rates in arrays and loop through the slabs. For S slabs, the calculation takes O(S) time per bill and O(S) extra storage for the arrays. Fixed if-statements take O(1) time for a fixed number of slabs and use no array storage. The array/loop design is easier to extend or revise when the number of slabs changes.

TASK 10 – Compare Expectations with Execution (4 Marks)

1. Aggregate totals

Using the five sample values, the exact paise total is Rs. 1,487.50. For 4,000 bills, the maximum possible total depends on the maximum monthly usage per household; it cannot be stated without a usage limit. The same is true for 60,000 and 800,000 bills. Therefore, select a 64-bit accumulator and verify that the assumed maximum bill multiplied by the number of households fits.

If a representative bill were Rs. 1,000, totals would be Rs. 40,00,000 for 4,000 bills, Rs. 6,00,00,000 for 60,000 bills and Rs. 80,00,00,000 for 800,000 bills. These are illustrative estimates, not maximum bounds.

2. Type limits and system sizes

- A 16-bit unsigned short can represent 0–65535. It is suitable for the original meter register, not for a district-wide total.
- int and long sizes depend on the implementation. Check sizeof and limits rather than assuming a size.
- long long is at least 64 bits in standard C and is suitable for a large paise accumulator after checking the maximum expected bill.

3. Double versus integer paise

A double accumulator has a wide range and good precision, but it is not an exact representation of every decimal currency value. Integer paise produces exact currency totals and makes rounding policy clear. Addition of either type is O(1) time and each accumulator is O(1) space. Integer paise is recommended for financial totals; use a sufficiently wide signed integer and check overflow before adding.

4. Why observed output may differ

Differences can result from using float, integer division, incorrect format specifiers, narrowing casts, or an overflowed accumulator. A correct implementation uses integer paise, 64-bit totals, explicit rollover handling and matching format specifiers.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

1. Recommended original design

- Use unsigned short (or uint16_t) for the original 16-bit meter register, and a wider unsigned type for consumed units.
- Use long long / int64_t for money in paise and district totals. Do not store money as float.
- Use named tariff constants in one header or configuration module. Use safe functions for billing; use macros only for simple compile-time constants and carefully parenthesised expressions.
- Validate all input, check scan results, detect suspicious meter resets, and test slab boundaries and rollover.

2. Tariff revision and limitations

For the original fixed tariff, change the central tariff definitions once and rebuild. The original design assumes whole-unit readings, one rollover at most per billing interval, one tariff category and a fixed monthly tariff. It needs modification for smart-meter intervals, time-of-day rates, fractional units and runtime tariff changes.

3. District-scale correctness

The billing calculation for each household remains correct at district scale if the inputs are valid, but aggregate totals must use a sufficiently wide checked accumulator. The program should also process records in batches or stream them rather than holding every bill in memory. Per-household billing is O(1) for a fixed number of slabs; processing N households is O(N).

Alternative: runtime configuration file

Reading tariff values from a configuration file allows revisions without recompiling. The program pays a small one-time startup cost to read and validate the file, then uses the loaded values for each bill. The configuration occupies O(S) memory for S slabs. Recompiling and redistributing a program after every tariff change takes deployment effort and can create version mismatch. Runtime configuration is preferable when tariffs change often, provided the file is validated and securely managed.

TASK 12 – Re-evaluate and Adapt the Design: Smart-Meter Challenge (8 Marks)

Changed requirements: smart meters report watt-hours every 15 minutes, there are 35,040 readings per household per year, usage between 6 PM and 10 PM costs 20% more, and tariff values must be changeable without recompilation.

A. Impact analysis (2 Marks)

- The input is now watt-hours (Wh), not a whole-unit cumulative meter register. Monthly/yearly energy can be fractional in kWh, and a household may produce 35,040 interval readings per year.
- A 16-bit reading type is no longer suitable for annual/monthly energy totals. Use uint64_t for accumulated Wh and int64_t for money in paise.
- Tariff values should not be fixed only with #define. Load them from a validated configuration file at program start.
- The billing system must identify whether each interval falls between 18:00 and 22:00 and apply the 20% peak rate to that interval.

B. Evaluate the original design (2 Marks)

The original unsigned-short rollover function remains useful only for the old 16-bit register. It cannot represent a year of interval energy. The original whole-unit slab calculation loses fractional kWh if each interval is rounded separately. A 64-bit Wh accumulator avoids this loss. The original compile-time macros are not suitable for tariffs that must change

without recompilation.

There are 365 × 24 × 4 = 35,040 quarter-hour readings per year. If each reading is stored as a 64-bit value, the raw energy values alone use 35,040 × 8 = 280,320 bytes per household per year (about 273.75 KiB), excluding timestamps and metadata. A streaming calculation can instead process one reading at a time with O(1) working memory.

C. Adapted design and working code (2 Marks)

The following illustrative implementation accepts interval energy in Wh and a peak/off-peak flag. It allocates each interval's energy through the monthly cumulative slabs, so slab boundaries are applied to the actual accumulated energy. Peak intervals receive 20% higher rates. The fixed charge is added once per bill. Rates are read as integer paise per kWh from a configuration file.

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

/* Example tariff.cfg:
   fixed_paise 5000
   rate2_paise_per_kwh 225
   rate3_paise_per_kwh 450
*/
typedef struct {
    int64_t fixed_paise;
    int64_t rate2;
    int64_t rate3;
} Tariff;

int load_tariff(const char *path, Tariff *t)
{
    FILE *f = fopen(path, "r");
    if (!f) return 0;
    int ok = fscanf(f, "fixed_paise %" SCNd64
                       " rate2_paise_per_kwh %" SCNd64
                       " rate3_paise_per_kwh %" SCNd64,
                    &t->fixed_paise, &t->rate2, &t->rate3) == 3;
    fclose(f);
    return ok && t->fixed_paise >= 0 && t->rate2 >= 0 && t->rate3 >= 0;
}

/* Add one interval. total_wh is cumulative energy before this interval.
   peak is 1 from 18:00 inclusive to 22:00 exclusive. */
int add_interval(uint64_t *total_wh, int64_t *energy_paise,
                 uint64_t interval_wh, int peak, const Tariff *t)
{
    uint64_t left = interval_wh;
    const uint64_t slab1 = 100000ULL; /* 100 kWh in Wh */
    const uint64_t slab2_end = 200000ULL;

    while (left > 0) {
        uint64_t pos = *total_wh;
        uint64_t take;
        int64_t rate;

        if (pos < slab1) {
            take = slab1 - pos; rate = 0;
        } else if (pos < slab2_end) {
            take = slab2_end - pos; rate = t->rate2;
        } else {
            take = left; rate = t->rate3;
        }
        if (take > left) take = left;

        if (peak) rate = (rate * 120 + 50) / 100; /* nearest paise rate */
        /* Wh * paise/kWh / 1000; keep numerator exact until conversion. */
        *energy_paise += (int64_t)(take * (uint64_t)rate / 1000ULL);
        *total_wh += take;
        left -= take;
    }
    return 1;
}

int main(void)
{
    Tariff tariff;
    uint64_t total_wh = 0;
    int64_t energy_paise = 0;

    if (!load_tariff("tariff.cfg", &tariff)) {
```

```
        puts("Invalid or missing tariff configuration.");
        return 1;
    }

    /* Example: 2 kWh off-peak and 1 kWh peak. */
    add_interval(&total_wh, &energy_paise, 2000, 0, &tariff);
    add_interval(&total_wh, &energy_paise, 1000, 1, &tariff);

    int64_t total = tariff.fixed_paise + energy_paise;
    printf("Energy: %" PRIu64 " Wh\\n", total_wh);
    printf("Bill: Rs. %" PRId64 ".%02" PRId64 "\\n",
           total / 100, total % 100);
    return 0;
}
```

Implementation note: for a production program, use checked arithmetic for multiplication and addition, and retain any fractional paise remainder (or accumulate the exact numerator in a wider integer) rather than truncating each interval. The example shows the revised data flow and tariff selection; a production version should also parse a robust configuration format and validate timestamps, duplicate readings and missing intervals.

D. Evidence-based defence (2 Marks)

- The revised design is a **redesign** of the calculation core: 64-bit Wh accumulation replaces the 16-bit register difference; the tariff is loaded at run time; and peak/off-peak status is applied per interval.
- A quarter-hour interval is classified as peak when its local time is from 18:00 inclusive to 22:00 exclusive. The peak energy rate is 1.2 times the corresponding base rate.
- The calculation still follows the cumulative slab structure: first 100 kWh free, next 100 kWh at the second rate, and energy above 200 kWh at the third rate. Fractional kWh is retained as Wh until conversion to paise.
- The system processes 35,040 readings per year in O(R) time for R readings. A streaming design needs O(1) working memory per household, excluding any audit log. A configuration file needs O(S) memory for S tariff values and avoids recompilation when rates change.

For an exact production bill, calculate energy charge in a wide integer numerator (Wh × paise/kWh), accumulate it, and divide by 1000 once at the end using a documented rounding rule. This avoids losing fractional paise at every 15-minute interval. The final bill must be checked against the maximum possible annual Wh and tariff before selecting accumulator limits.

Conclusion

The recommended original billing program uses unsigned 16-bit readings only for the original meter register, integer paise for exact money, wide integer totals, centralised tariff constants, safe functions, validated input and boundary testing. For the smart-meter requirement, the correct decision is to redesign the calculation around 64-bit watt-hour accumulation, interval-based peak pricing and runtime tariff configuration. These choices improve correctness, maintainability and scalability.

GitHub submission checklist

- Create one repository named using the required format: CSA02--Assignment1.
- Create separate folders for Task01 through Task12. Keep each source file, input file, output screenshot and short result note in its task folder.
- Add README.md with student name, register number, compiler/version, build commands and a brief result for every task.
- Commit progressively, at least one commit per task. Include the working repository link on the report cover page.
- Compile with warnings enabled: gcc -Wall -Wextra -std=c11 file.c -o program. Keep the exact commands and test evidence.

Note: Fill in your name, register number and working GitHub repository link before submission. Verify the code on your own compiler and retain screenshots of actual execution; do not label an unexecuted result as measured evidence.